LAPORAN PRAKTIKUM MODUL 3

FUNCTION & ARRAY

DISUSUN OLEH :

SAQYA RAHMATILLAH
2025573010131

Kelas : TI 1C
Mata Kuliah : Praktikum Algoritma dan Struktur Data
Jurusan : Teknologi Informasi dan Komputer
Prodi : Teknik Informatika
Dosen : Muhammad Reza Zulman, S.ST.,M.Sc

PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER

POLITEKNIK NEGERI LHOKSEUMAWE

2026

KATA PENGANTAR

Puji syukur saya panjatkan kehadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya saya dapat menyelesaikan laporan praktikum Modul 3 yang berjudul *Function & Array* dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas pada mata kuliah Struktur Data dan Algoritma. Dalam praktikum ini, saya mempelajari konsep dasar yang sangat penting dalam pemrograman JavaScript, yaitu function dan array. Selain itu, saya juga mempelajari penggunaan arrow function, callback, serta berbagai method array yang mempermudah pengolahan data.

Saya menyadari bahwa laporan ini masih memiliki kekurangan, oleh karena itu saya mengharapkan kritik dan saran yang membangun agar laporan ini dapat menjadi lebih baik di masa yang akan datang.

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, kemampuan untuk menulis kode yang rapi, terstruktur, dan mudah dipahami sangatlah penting. Jika program dibuat tanpa struktur yang jelas, maka kode akan sulit dibaca dan sulit dikembangkan di kemudian hari. Oleh karena itu, dibutuhkan konsep seperti function agar kode dapat dibagi menjadi bagian-bagian kecil yang memiliki tugas masing-masing.

Function membantu programmer untuk menghindari penulisan kode yang berulang-ulang. Dengan adanya function, kita cukup menulis satu kali lalu bisa digunakan berkali-kali. Hal ini membuat program menjadi lebih efisien dan mudah dikelola.

Selain function, array juga merupakan struktur data yang sangat penting. Dalam banyak kasus, kita tidak hanya bekerja dengan satu data saja, tetapi dengan banyak data sekaligus. Array memungkinkan kita untuk menyimpan banyak nilai dalam satu variabel dan mengolahnya dengan lebih mudah.

JavaScript juga menyediakan berbagai method array seperti *map*, *filter*, dan *reduce* yang sangat membantu dalam pengolahan data. Dengan menggunakan method tersebut, kita dapat menulis kode yang lebih singkat dan mudah dipahami dibandingkan menggunakan perulangan biasa.

Oleh karena itu, pada praktikum ini mahasiswa mempelajari konsep function dan array secara lebih mendalam agar dapat digunakan dalam pembuatan program yang lebih kompleks di masa depan.

1.2 Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

- Mendefinisikan dan memanggil function dengan parameter dan nilai kembalian (return value)
- Memahami konsep scope variabel di dalam dan di luar function
- Menulis arrow function sebagai sintaks modern pengganti function biasa
- Memahami konsep callback dan penggunaannya sebagai argumen function
- Membuat dan memanipulasi array menggunakan berbagai method bawaan
- Menggunakan method iterasi array: forEach, map, filter, dan reduce
- Memahami rekursi sebagai teknik function yang memanggil dirinya sendiri

BAB II

DASAR TEORI

2.1 Function

Function adalah blok kode yang diberi nama dan dapat dipanggil (dieksekusi) kapan saja dan berulang kali. Bayangkan function seperti sebuah mesin atau resep — kamu mendefinisikannya sekali, lalu bisa menggunakannya berkali-kali tanpa perlu menulis ulang kodenya.

Mengapa function penting? Prinsip utama pemrograman adalah DRY (Don't Repeat Yourself). Daripada menulis kode yang sama di banyak tempat, cukup bungkus kode itu dalam sebuah function.

Anatomi sebuah function:

- Parameter — Variabel input yang diterima function dari luar. Didefinisikan di dalam kurung().
- Return value — Nilai yang dikembalikan function setelah selesai dieksekusi, menggunakan kata kunci return.
- Scope — Variabel yang dideklarasikan di dalam function hanya bisa diakses di dalam function itu saja (local scope). Variabel di luar function disebut global scope.

Perlu diingat: setelah JavaScript menemukan kata kunci return, eksekusi function langsung berhenti. Kode di bawah return tidak akan pernah dijalankan.

2.2 Arrow Function & Callback

Arrow function adalah sintaks penulisan function yang lebih ringkas, diperkenalkan di ES6 (2015). Alih-alih menulis function nama(params) { ... }, kita bisa menulis const nama = (params) => { ... }.

Aturan penulisan arrow function:

- Jika hanya ada satu parameter, kurung bisa dihilangkan: x => x * 2
- Jika body hanya satu baris dan langsung return, kurung kurawal dan return bisa dihilangkan: (a, b) => a + b
- Jika body lebih dari satu baris, tetap gunakan kurung kurawal dan return.

Callback adalah sebuah function yang dikirimkan sebagai argumen ke function lain. Function penerima akan memanggil callback tersebut di waktu yang tepat. Ini adalah konsep yang sangat fundamental di JavaScript — hampir semua event handling, timer, dan method array menggunakan callback.

Analogi callback: Bayangkan kamu memesan makanan di restoran. Kamu memberikan nomor mejamu (callback) ke pramusaji. Setelah makanan siap (event), pramusaji memanggil nomor mejamu. Kamu tidak tahu kapan tepatnya dipanggil, tapi kamu sudah siapkan respon.

2.3 Array Dasar

Array adalah struktur data yang menyimpan kumpulan nilai dalam satu variabel, berurutan dan terindeks. Bayangkan array seperti sebuah rak buku bernomor — setiap buku (elemen) memiliki posisi (indeks) yang dimulai dari angka 0.

Contoh: const buah = ['Apel', 'Mangga', 'Jeruk']

- buah[0] → 'Apel' (indeks pertama = 0)
- buah[1] → 'Mangga' (indeks kedua = 1)
- buah[2] → 'Jeruk' (indeks ketiga = 2)
- buah.length → 3 (jumlah elemen)

Array di JavaScript bersifat dinamis — ukurannya bisa berubah sewaktu-waktu. Array juga bisa menyimpan tipe data yang berbeda-beda dalam satu array (meskipun praktik terbaik Adalah menyimpan satu jenis data per array).

Method bawaan array yang sering digunakan untuk manipulasi:

- push(x) — tambah elemen x di akhir array
- pop() — hapus dan kembalikan elemen terakhir
- unshift(x) — tambah elemen x di awal array
- shift() — hapus dan kembalikan elemen pertama
- splice(i, n) — hapus n elemen mulai indeks i
- slice(a, b) — salin elemen dari indeks a sampai sebelum b (non-destructive)
- indexOf(x) — cari indeks elemen x (-1 jika tidak ditemukan)
- includes(x) — cek apakah x ada di array (mengembalikan boolean)

2.4 Method Iterasi Array

JavaScript menyediakan method iterasi array yang powerful. Method-method ini menerima sebuah callback function sebagai argumen, dan secara otomatis memanggil callback untuk setiap elemen array.

- `forEach(callback)` — Menjalankan callback untuk setiap elemen. Tidak mengembalikan nilai apapun. Digunakan hanya untuk efek samping (side effects) seperti mencetak atau menyimpan ke database.
- `map(callback)` — Membuat array baru dengan menerapkan callback ke setiap elemen. Array asli tidak berubah. Digunakan untuk mentransformasi setiap elemen.
- `filter(callback)` — Membuat array baru yang hanya berisi elemen yang memenuhi kondisi (callback mengembalikan true). Array asli tidak berubah. Digunakan untuk menyaring elemen.
- `reduce(callback, nilaiAwal)` — Mereduksi seluruh array menjadi satu nilai tunggal. Callback menerima dua argumen: accumulator (nilai yang terakumulasi) dan currentValue (elemen saat ini).

Ingat: map dan filter selalu menghasilkan array baru, sedangkan reduce menghasilkan satu nilai (angka, string, object, dll).

2.5 Rekursi

Rekursi adalah teknik di mana sebuah function memanggil dirinya sendiri untuk memecahkan masalah yang lebih kecil secara bertahap, sampai mencapai kondisi berhenti.

Setiap function rekursif harus memiliki dua bagian:

- Base case (kondisi berhenti) — kondisi di mana function tidak lagi memanggil dirinya sendiri. Tanpa ini, rekursi akan berjalan selamanya dan menyebabkan error Maximum call stack exceeded.

- Recursive case — bagian di mana function memanggil dirinya sendiri dengan input yang lebih kecil/sederhana.

Contoh: Menghitung faktorial dari n (n!):

- $5! = 5 \times 4! = 5 \times 4 \times 3! = 5 \times 4 \times 3 \times 2! = 5 \times 4 \times 3 \times 2 \times 1! = 120$
- Base case: faktorial(1) = 1
- Recursive case: faktorial(n) = n × faktorial(n-1)

Rekursi sangat penting dalam Struktur Data dan Algoritma karena banyak algoritma pada pohon (tree), graf (graph), dan pencarian (searching) yang paling mudah diimplementasikan secara rekursif.

BAB III

LANGKAH-LANGKAH PRAKTIKUM

3.1 Persiapan Project

1. Buka Git Bash. Kemudian navigasikan ke folder repository GitHub kamu yang sudah di-clone.

2. Buat folder praktikum-3/ dan masuk ke dalamnya.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData (main)
$ mkdir praktikum-3
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData (main)
$ cd praktikum-3
```

3. Inisialisasi npm project.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-3 (main)
$ npm init -y
Wrote to C:\Users\USER\2025573010131_PratikumAlgoritmaDanStrukturData\praktikum-3\package.json:

{
  "name": "praktikum-3",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

3.2 Praktik: Function

1. Pastikan kamu berada di dalam folder praktikum-3/. Buat file baru bernama 01-function-dasar.js.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-3 (main)
$ touch 01-function-dasar.js
```

2. Buka file 01-function-dasar.js di VS Code, lalu ketikkan kode berikut.

```javascript
function sapa() {
  console.log("Halo! Selamat datang di praktikum JavaScript.");
}
sapa();
sapa();
function sapaNama(nama) {
  console.log(`Halo, ${nama}! Selamat belajar.`);
}
sapaNama("Budi");
sapaNama("Siti");

function tambah(angka1, angka2) {
  const hasil = angka1 + angka2;
  return hasil;
}
const hasilPenjumlahan = tambah(10, 25);
console.log("10 + 25 =", hasilPenjumlahan);
console.log("7 + 13 =", tambah(7, 13));

function hitung(nilai, pengali = 2) {
  return nilai * pengali;
}
console.log(hitung(5));
console.log(hitung(5, 3));

const pesanGlobal = "Saya ada di mana saja";
function cekScope() {
  const pesanLokal = "Saya hanya ada di dalam function ini";
  console.log(pesanGlobal);
  console.log(pesanLokal);
}
cekScope();
console.log(pesanGlobal);
```

3. Simpan file, lalu jalankan dari terminal.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-3 (main)
$ node 01-function-dasar.js
Halo! Selamat datang di praktikum JavaScript.
Halo! Selamat datang di praktikum JavaScript.
Halo, Budi! Selamat belajar.
Halo, Siti! Selamat belajar.
10 + 25 = 35
7 + 13 = 20
10
15
Saya ada di mana saja
Saya hanya ada di dalam function ini
Saya ada di mana saja
```

4. Latihan 1: Function Kalkulator Lengkap

```javascript
// Latihan 1: Function Kalkulator Lengkap
// Function penjumlahan (misal sudah ada)
function tambah(a, b) {
  return a + b;
}

// Function pengurangan
function kurang(a, b) {
  return a - b;
}

// Function perkalian
function kali(a, b) {
  return a * b;
}

// Function pembagian
function bagi(a, b) {
  if (b === 0) {
    console.log("Error: tidak bisa membagi dengan nol");
    return null;
  }
  return a / b;
}

// Function kalkulator
function kalkulator(a, operator, b) {
  switch (operator) {
    case "+":
      return tambah(a, b);
    case "-":
      return kurang(a, b);
    case "*":
      return kali(a, b);
    case "/":
      return bagi(a, b);
    default:
      console.log("Operator tidak dikenal");
      return null;
  }
}

// Pengujian (minimal 5)
console.log(kalkulator(10, "+", 5)); // 15
console.log(kalkulator(10, "-", 5)); // 5
console.log(kalkulator(10, "*", 5)); // 50
console.log(kalkulator(10, "/", 5)); // 2
console.log(kalkulator(10, "/", 0)); // Error + null
```

5. Simpan dan jalankan: node 01-function-dasar.js.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-3 (main)
$ node 01-function-dasar.js
Halo! Selamat datang di praktikum JavaScript.
Halo! Selamat datang di praktikum JavaScript.
Halo, Budi! Selamat belajar.
Halo, Siti! Selamat belajar.
10 + 25 = 35
7 + 13 = 20
10
15
Saya ada di mana saja
Saya hanya ada di dalam function ini
Saya ada di mana saja
15
5
50
2
Error: tidak bisa membagi dengan nol
null
```

3.3 Praktik: Arrow Function & Callback

1. Buat file baru bernama 02-arrow-callback.js di dalam folder praktikum-3/.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-3 (main)
$ touch 02-arrow-callback.js
```

2. Buka file 02-arrow-callback.js di VS Code, lalu ketikkan kode berikut.

```javascript
function kuadratBiasa(x) {
  return x * x;
}

const kuadrat = (x) => {
  return x * x;
};

const kuadratRingkas = (x) => x * x;
console.log("=== Perbandingan Penulisan ===");
console.log("Biasa :", kuadratBiasa(5));
console.log("Arrow :", kuadrat(5));
console.log("Ringkas :", kuadratRingkas(5));

const luas = (panjang, lebar) => panjang * lebar;
const salam = (nama, waktu) => `Selamat ${waktu}, ${nama}!`;
console.log("Luas 4x6 :", luas(4, 6));
console.log(salam("Budi", "Pagi"));

function lakukanOperasi(angka, operasiCallback) {
  console.log(`Angka awal: ${angka}`);
  const hasil = operasiCallback(angka);
  console.log(`Hasil setelah operasi: ${hasil}`);
}
console.log("\n=== Callback ===");
lakukanOperasi(7, kuadratRingkas);
lakukanOperasi(10, (x) => x + 100);
lakukanOperasi(20, function (x) {
  return x / 2;
});

console.log("\n=== setTimeout (Callback) ===");
console.log("Pesan 1: Sebelum timer");
setTimeout(() => {
  console.log("Pesan 3: Ini dari dalam setTimeout (setelah 1 detik)");
}, 1000);
console.log("Pesan 2: Setelah mendaftarkan timer");
```

3. Simpan file, lalu jalankan dari terminal.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-3 (main)
$ node 02-arrow-callback.js
=== Perbandingan Penulisan ===
Biasa : 25
Arrow : 25
Ringkas : 25
Luas 4x6 : 24
Selamat Pagi, Budi!

=== Callback ===
Angka awal: 7
Hasil setelah operasi: 49
Angka awal: 10
Hasil setelah operasi: 110
Angka awal: 20
Hasil setelah operasi: 10

=== setTimeout (Callback) ===
Pesan 1: Sebelum timer
Pesan 2: Setelah mendaftarkan timer
Pesan 3: Ini dari dalam setTimeout (setelah 1 detik)
```

4. Latihan 2: Pipeline Transformasi Data

```javascript
// Latihan 2: Pipeline Transformasi Data
// Arrow function huruf besar
const keHurufBesar = (str) => {
  return str.toUpperCase();
};

// Arrow function tambah tanda seru
const tambahSeru = (str) => {
  return str + "!!!";
};

// Arrow function hitung jumlah kata
const hitungKata = (str) => {
  return str.split(" ").length;
};

// Function proses kalimat dengan callback
function prosesKalimat(kalimat, transformasiCallback) {
  const hasil = transformasiCallback(kalimat);
  console.log(hasil);
}

// Pengujian
const kalimat = "belajar javascript itu menyenangkan";

prosesKalimat(kalimat, keHurufBesar);
prosesKalimat(kalimat, tambahSeru);
prosesKalimat(kalimat, hitungKata);
```

5. Simpan dan jalankan. Pastikan outputnya sesuai harapan.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-3 (main)
$ node 02-arrow-callback.js
=== Perbandingan Penulisan ===
Biasa : 25
Arrow : 25
Ringkas : 25
Luas 4x6 : 24
Selamat Pagi, Budi!

=== Callback ===
Angka awal: 7
Hasil setelah operasi: 49
Angka awal: 10
Hasil setelah operasi: 110
Angka awal: 20
Hasil setelah operasi: 10

=== setTimeout (Callback) ===
Pesan 1: Sebelum timer
Pesan 2: Setelah mendaftarkan timer
BELAJAR JAVASCRIPT ITU MENYENANGKAN
belajar javascript itu menyenangkan!!!
4
Pesan 3: Ini dari dalam setTimeout (setelah 1 detik)
```

3.4 Praktik: Array Dasar

1. Buat file baru bernama 03-array-dasar.js di dalam folder praktikum-3/.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-3 (main)
$ touch 03-array-dasar.js
```

2. Buka file 03-array-dasar.js di VS Code, lalu ketikkan kode berikut.

```javascript
const mahasiswa = ["Budi", "Siti", "Ahmad", "Rina"];
const nilai = [85, 92, 78, 95, 88];

console.log("=== Array Awal ===");
console.log("Mahasiswa:", mahasiswa);
console.log("Nilai :", nilai);
console.log("Jumlah mahasiswa:", mahasiswa.length);

console.log("\n=== Akses Elemen ===");
console.log("Elemen pertama :", mahasiswa[0]); // indeks 0
console.log("Elemen ketiga :", mahasiswa[2]); // indeks 2
console.log("Elemen terakhir:", mahasiswa[mahasiswa.length - 1]);

mahasiswa[1] = "Siti Rahayu";
console.log("\nSetelah diubah:", mahasiswa);

console.log("\n=== Manipulasi Array ===");
mahasiswa.push("Doni");
console.log("Setelah push :", mahasiswa);
mahasiswa.unshift("Zahra");
console.log("Setelah unshift :", mahasiswa);
const dihapusAkhir = mahasiswa.pop();
console.log("Dihapus (pop) :", dihapusAkhir);
console.log("Setelah pop :", mahasiswa);
const dihapusAwal = mahasiswa.shift();
console.log("Dihapus (shift) :", dihapusAwal);
console.log("Setelah shift :", mahasiswa);

console.log("\n=== Pencarian ===");
console.log("Indeks Ahmad :", mahasiswa.indexOf("Ahmad"));
console.log("Ada Rina? :", mahasiswa.includes("Rina"));
console.log("Ada Budi? :", mahasiswa.includes("Budi"));

const sebagian = nilai.slice(1, 4);
console.log("\nNilai asli :", nilai);
console.log("Slice [1,4] :", sebagian);
```

3. Simpan file, lalu jalankan dari terminal.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-3 (main)
$ node 03-array-dasar.js
=== Array Awal ===
Mahasiswa: [ 'Budi', 'Siti', 'Ahmad', 'Rina' ]
Nilai : [ 85, 92, 78, 95, 88 ]
Jumlah mahasiswa: 4

=== Akses Elemen ===
Elemen pertama : Budi
Elemen ketiga : Ahmad
Elemen terakhir: Rina

Setelah diubah: [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]

=== Manipulasi Array ===
Setelah push : [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina', 'Doni' ]
Setelah unshift : [ 'Zahra', 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina', 'Doni' ]
Dihapus (pop) : Doni
Setelah pop : [ 'Zahra', 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]
Dihapus (shift) : Zahra
Setelah shift : [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]

=== Pencarian ===
Indeks Ahmad : 2
Ada Rina? : true
Ada Budi? : true

Nilai asli : [ 85, 92, 78, 95, 88 ]
Slice [1,4] : [ 92, 78, 95 ]
```

4. Latihan 3: Manajemen Daftar Belanja

```javascript
// Latihan 3: Manajemen Daftar Belanja
// Membuat array daftarBelanja (5 item)
let daftarBelanja = ["Beras", "Gula", "Minyak", "Telur", "Garam"];

// Menampilkan isi daftar dengan nomor urut
console.log("Daftar Belanja:");
for (let i = 0; i < daftarBelanja.length; i++) {
  console.log(i + 1 + ". " + daftarBelanja[i]);
}

// Menambahkan 2 item baru
daftarBelanja.push("Susu");
daftarBelanja.push("Kopi");

console.log("\nSetelah ditambah:");
console.log(daftarBelanja);

// Menghapus item pertama
let itemDihapus = daftarBelanja.shift();
console.log("\nItem yang dihapus:", itemDihapus);

// Cek apakah 'Susu' ada
let adaSusu = daftarBelanja.includes("Susu");
if (adaSusu) {
  console.log("\nSusu ada di daftar belanja");
} else {
  console.log("\nSusu tidak ada di daftar belanja");
}

// Menampilkan jumlah item akhir
console.log("\nJumlah item akhir:", daftarBelanja.length);
```

5. Simpan dan jalankan.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-3 (main)
$  node 03-array-dasar.js
=== Array Awal ===
Mahasiswa: [ 'Budi', 'Siti', 'Ahmad', 'Rina' ]
Nilai : [ 85, 92, 78, 95, 88 ]
Jumlah mahasiswa: 4

=== Akses Elemen ===
Elemen pertama : Budi
Elemen ketiga : Ahmad
Elemen terakhir: Rina

Setelah diubah: [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]

=== Manipulasi Array ===
Setelah push : [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina', 'Doni' ]
Setelah unshift : [ 'Zahra', 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina', 'Doni' ]
Dihapus (pop) : Doni
Setelah pop : [ 'Zahra', 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]
Dihapus (shift) : Zahra
Setelah shift : [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]
```

```
=== Pencarian ===
Indeks Ahmad : 2
Ada Rina? : true
Ada Budi? : true

Nilai asli : [ 85, 92, 78, 95, 88 ]
Slice [1,4] : [ 92, 78, 95 ]
Daftar Belanja:
1. Beras
2. Gula
3. Minyak
4. Telur
5. Garam

Setelah ditambah:
[
  'Beras',  'Gula',
  'Minyak', 'Telur',
  'Garam',  'Susu',
  'Kopi'
]

Item yang dihapus: Beras

Susu ada di daftar belanja

Jumlah item akhir: 6
```

3.5 Praktik: Method Iterasi Array

1. Buat file baru bernama 04-array-method.js di dalam folder praktikum-3/.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-3 (main)
$ touch 04-array-method.js
```

2. Buka file 04-array-method.js di VS Code, lalu ketikkan kode berikut.

```javascript
const nilaiMahasiswa = [75, 90, 55, 82, 68, 95, 48, 77];

console.log("=== forEach: Tampilkan Semua Nilai ===");
nilaiMahasiswa.forEach((nilai, indeks) => {
  console.log(` Mahasiswa-${indeks + 1}: ${nilai}`);
});

const gradeHuruf = nilaiMahasiswa.map((nilai) => {
  if (nilai >= 90) return "A";
  if (nilai >= 80) return "B";
  if (nilai >= 70) return "C";
  if (nilai >= 60) return "D";
  return "E";
});
console.log("\n=== map: Nilai ke Grade ===");
console.log("Nilai :", nilaiMahasiswa);
console.log("Grade :", gradeHuruf);

const nilaiLulus = nilaiMahasiswa.filter((nilai) => nilai >= 60);
const nilaiGagal = nilaiMahasiswa.filter((nilai) => nilai < 60);
console.log("\n=== filter: Lulus dan Tidak Lulus ===");
console.log("Semua nilai :", nilaiMahasiswa);
console.log("Nilai lulus :", nilaiLulus);
console.log("Nilai gagal :", nilaiGagal);

const totalNilai = nilaiMahasiswa.reduce((akumulator, nilai) => {
  return akumulator + nilai;
}, 0);
const rataRata = totalNilai / nilaiMahasiswa.length;
console.log("\n=== reduce: Statistik Nilai ===");
console.log("Total nilai :", totalNilai);
console.log("Rata-rata :", rataRata.toFixed(2));
const rataRataNilaiLulus = nilaiMahasiswa
  .filter((nilai) => nilai >= 60)
  .reduce((acc, val, idx, arr) => {
    return acc + val / arr.length;
  }, 0);
console.log("\n=== Method Chaining ===");
console.log("Rata-rata nilai lulus:", rataRataNilaiLulus.toFixed(2));
```

3. Simpan file, lalu jalankan dari terminal.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-3 (main)
$ node 04-array-method.js
=== forEach: Tampilkan Semua Nilai ===
 Mahasiswa-1: 75
 Mahasiswa-2: 90
 Mahasiswa-3: 55
 Mahasiswa-4: 82
 Mahasiswa-5: 68
 Mahasiswa-6: 95
 Mahasiswa-7: 48
 Mahasiswa-8: 77

=== map: Nilai ke Grade ===
Nilai : [
  75, 90, 55, 82,
  68, 95, 48, 77
]
Grade : [
  'C', 'A', 'E',
  'B', 'D', 'A',
  'E', 'C'
]

=== filter: Lulus dan Tidak Lulus ===
Semua nilai : [
  75, 90, 55, 82,
  68, 95, 48, 77
]
Nilai lulus : [ 75, 90, 82, 68, 95, 77 ]
Nilai gagal : [ 55, 48 ]
```

```
=== reduce: Statistik Nilai ===
Total nilai : 590
Rata-rata : 73.75

=== Method Chaining ===
Rata-rata nilai lulus: 81.17
```

4. Latihan 4: Analisis Data Produk

```javascript
// Latihan 4: Analisis Data Produk
// Deklarasi array produk
const produk = [
  { nama: "Laptop", harga: 8500000, stok: 5 },
  { nama: "Mouse", harga: 150000, stok: 0 },
  { nama: "Keyboard", harga: 450000, stok: 12 },
  { nama: "Monitor", harga: 3200000, stok: 0 },
  { nama: "Headset", harga: 350000, stok: 8 },
];

// Filter produk yang tersedia (stok > 0)
const produkTersedia = produk.filter((item) => item.stok > 0);
console.log("Produk tersedia:");
console.log(produkTersedia);

// Map untuk mengambil nama produk saja
const namaProduk = produkTersedia.map((item) => item.nama);
console.log("\nNama produk tersedia:");
console.log(namaProduk);

// Reduce untuk total nilai inventaris
const totalInventaris = produkTersedia.reduce((total, item) => {
  return total + item.harga * item.stok;
}, 0);
console.log("\nTotal nilai inventaris:", totalInventaris);

// forEach untuk menampilkan semua produk
console.log("\nDaftar produk:");
produk.forEach((item) => {
  if (item.stok > 0) {
    console.log(
      `[TERSEDIA] ${item.nama} - Rp${item.harga.toLocaleString()} (${item.stok} unit)`,
    );
  } else {
    console.log(`[HABIS] ${item.nama} - Rp${item.harga.toLocaleString()}`);
  }
});
```

5. Simpan dan jalankan.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-3 (main)
$ node 04-array-method.js
=== forEach: Tampilkan Semua Nilai ===
 Mahasiswa-1: 75
 Mahasiswa-2: 90
 Mahasiswa-3: 55
 Mahasiswa-4: 82
 Mahasiswa-5: 68
 Mahasiswa-6: 95
 Mahasiswa-7: 48
 Mahasiswa-8: 77

=== map: Nilai ke Grade ===
Nilai : [
  75, 90, 55, 82,
  68, 95, 48, 77
]
Grade : [
  'C', 'A', 'E',
  'B', 'D', 'A',
  'E', 'C'
]

=== filter: Lulus dan Tidak Lulus ===
Semua nilai : [
  75, 90, 55, 82,
  68, 95, 48, 77
]
Nilai lulus : [ 75, 90, 82, 68, 95, 77 ]
Nilai gagal : [ 55, 48 ]
```

```
=== reduce: Statistik Nilai ===
Total nilai : 590
Rata-rata : 73.75

=== Method Chaining ===
Rata-rata nilai lulus: 81.17
Produk tersedia:
[
  { nama: 'Laptop', harga: 8500000, stok: 5 },
  { nama: 'Keyboard', harga: 450000, stok: 12 },
  { nama: 'Headset', harga: 350000, stok: 8 }
]

Nama produk tersedia:
[ 'Laptop', 'Keyboard', 'Headset' ]

Total nilai inventaris: 50700000

Daftar produk:
[TERSEDIA] Laptop - Rp8.500.000 (5 unit)
[HABIS] Mouse - Rp150.000
[TERSEDIA] Keyboard - Rp450.000 (12 unit)
[HABIS] Monitor - Rp3.200.000
[TERSEDIA] Headset - Rp350.000 (8 unit)
```

3.6 Praktik: Rekursi

1. Buat file baru bernama 05-rekursi.js di dalam folder praktikum-3/.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-3 (main)
$ touch 05-rekursi.js
```

2. Buka file 05-rekursi.js di VS Code, lalu ketikkan kode berikut.

```javascript
function faktorial(n) {
  if (n <= 1) return 1;

  return n * faktorial(n - 1);
}
console.log("=== Faktorial ===");
console.log("0! =", faktorial(0));
console.log("1! =", faktorial(1));
console.log("5! =", faktorial(5));
console.log("7! =", faktorial(7));

function fibonacci(n) {
  if (n === 0) return 0;
  if (n === 1) return 1;
  return fibonacci(n - 1) + fibonacci(n - 2);
}
console.log("\n=== Fibonacci ===");
for (let i = 0; i <= 8; i++) {
  process.stdout.write(fibonacci(i) + " ");
}
console.log("");

function jumlahArray(arr, indeks = 0) {
  if (indeks === arr.length) return 0;
  return arr[indeks] + jumlahArray(arr, indeks + 1);
}
const angka = [3, 7, 2, 9, 5];
console.log("\n=== Jumlah Array dengan Rekursi ===");
console.log("Array :", angka);
console.log("Jumlah :", jumlahArray(angka));

function countdown(n) {
  if (n < 0) {
    console.log("Selesai!");
    return;
  }
  console.log(n);
  countdown(n - 1);
}
console.log("\n=== Countdown Rekursif ===");
countdown(5);

```

3. Simpan file, lalu jalankan dari terminal.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-3 (main)
$ node 05-rekursi.js
=== Faktorial ===
0! = 1
1! = 1
5! = 120
7! = 5040

=== Fibonacci ===
0 1 1 2 3 5 8 13 21

=== Jumlah Array dengan Rekursi ===
Array : [ 3, 7, 2, 9, 5 ]
Jumlah : 26

=== Countdown Rekursif ===
5
4
3
2
1
0
Selesai!
```

3.7 Tugas 1: Sistem Nilai Mahasiswa

1. Buat folder dan file tugas.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-3 (main)
$ mkdir tugas

USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-3 (main)
$ touch tugas/tugas-1.js tugas/tugas-2.js
```

2. Buka file tugas/tugas-1.js.

3. Untuk menyelesaikan latihan tersebut, ketikkan kode berikut

```javascript
// Data mahasiswa (minimal 6)
const dataMahasiswa = [
  { nama: "Andi", nilai: 85 },
  { nama: "Budi", nilai: 70 },
  { nama: "Citra", nilai: 90 },
  { nama: "Dina", nilai: 60 },
  { nama: "Eka", nilai: 75 },
  { nama: "Fajar", nilai: 50 },
];

// Function hitung statistik
function hitungStatistik(arrMahasiswa) {
  const hasil = arrMahasiswa.reduce(
    (acc, mhs) => {
      acc.total += 1;
      acc.jumlahNilai += mhs.nilai;
      if (mhs.nilai > acc.tertinggi) acc.tertinggi = mhs.nilai;
      if (mhs.nilai < acc.terendah) acc.terendah = mhs.nilai;
      return acc;
    },
    {
      total: 0,
      jumlahNilai: 0,
      tertinggi: -Infinity,
      terendah: Infinity,
    },
  );

```

```javascript
  return {
    total: hasil.total,
    rataRata: hasil.jumlahNilai / hasil.total,
    tertinggi: hasil.tertinggi,
    terendah: hasil.terendah,
  };
}

// Function filter lulus
function filterLulus(arrMahasiswa, batasLulus) {
  return arrMahasiswa.filter((mhs) => mhs.nilai >= batasLulus);
}

// Function tambah grade
function tambahGrade(arrMahasiswa) {
  return arrMahasiswa.map((mhs) => {
    let grade;
    if (mhs.nilai >= 85) grade = "A";
    else if (mhs.nilai >= 75) grade = "B";
    else if (mhs.nilai >= 65) grade = "C";
    else if (mhs.nilai >= 55) grade = "D";
    else grade = "E";

    return { ...mhs, grade };
  });
}

// Pemanggilan function
const statistik = hitungStatistik(dataMahasiswa);
const mahasiswaLulus = filterLulus(dataMahasiswa, 70);
const mahasiswaDenganGrade = tambahGrade(dataMahasiswa);

// Output statistik
console.log("=== Statistik Nilai Mahasiswa ===");
console.log(`Total Mahasiswa : ${statistik.total}`);
console.log(`Rata-rata Nilai : ${statistik.rataRata}`);
console.log(`Nilai Tertinggi : ${statistik.tertinggi}`);
console.log(`Nilai Terendah  : ${statistik.terendah}`);

// Output mahasiswa lulus
console.log("\n=== Mahasiswa Lulus (>= 70) ===");
mahasiswaLulus.forEach((mhs) => {
  console.log(`${mhs.nama} - ${mhs.nilai}`);
});

// Output semua mahasiswa dengan grade
console.log("\n=== Data Mahasiswa + Grade ===");
mahasiswaDenganGrade.forEach((mhs) => {
  console.log(`${mhs.nama} - Nilai: ${mhs.nilai}, Grade: ${mhs.grade}`);
});

```

4. Simpan dan jalankan.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-3 (main)
$ node tugas/tugas-1.js
=== Statistik Nilai Mahasiswa ===
Total Mahasiswa : 6
Rata-rata Nilai : 71.66666666666667
Nilai Tertinggi : 90
Nilai Terendah  : 50

=== Mahasiswa Lulus (>= 70) ===
Andi - 85
Budi - 70
Citra - 90
Eka - 75

=== Data Mahasiswa + Grade ===
Andi - Nilai: 85, Grade: A
Budi - Nilai: 70, Grade: C
Citra - Nilai: 90, Grade: A
Dina - Nilai: 60, Grade: D
Eka - Nilai: 75, Grade: B
Fajar - Nilai: 50, Grade: E
```

3.8 Tugas 2: Pangkat dan Palindrom Rekursif

1. Buka file tugas/tugas-2.js.

2. Untuk menyelesaikan latihan tersebut, ketikkan kode berikut

```javascript
// Function rekursif pangkat
function pangkat(basis, eksponen) {
  if (eksponen === 0) return 1; // base case
  return basis * pangkat(basis, eksponen - 1);
}

// Uji pangkat
console.log("=== Uji Pangkat ===");
console.log(`2^3 = ${pangkat(2, 3)}`);
console.log(`5^2 = ${pangkat(5, 2)}`);
console.log(`3^4 = ${pangkat(3, 4)}`);
console.log(`10^0 = ${pangkat(10, 0)}`);

// Function rekursif balik string
function balikString(str) {
  if (str.length <= 1) return str; // base case
  return str[str.length - 1] + balikString(str.slice(0, str.length - 1));
}

// Uji balik string
console.log("\n=== Uji Balik String ===");
console.log(`halo -> ${balikString("halo")}`);
console.log(`javascript -> ${balikString("javascript")}`);

// Function cek palindrom
function cekPalindrom(str) {
  const dibalik = balikString(str);
  return str === dibalik;
}

// Uji palindrom
console.log("\n=== Uji Palindrom ===");
const kata = ["katak", "civic", "level", "halo"];

kata.forEach((k) => {
  if (cekPalindrom(k)) {
    console.log(`${k} adalah palindrom`);
  } else {
    console.log(`${k} bukan palindrom`);
  }
});
```

3. Simpan dan jalankan.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-3 (main)
$ node tugas/tugas-2.js
=== Uji Pangkat ===
2^3 = 8
5^2 = 25
3^4 = 81
10^0 = 1

=== Uji Balik String ===
halo -> olah
javascript -> tpircsavaj

=== Uji Palindrom ===
katak adalah palindrom
civic adalah palindrom
level adalah palindrom
halo bukan palindrom
```

3.9 Langkah-langkah Push ke GitHub

1. Buka terminal Git Bash, kemudian masuk ke folder project dengan perintah:

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData (main)
$ cd praktikum-3
```

2. Tambahkan seluruh file dengan perintah:

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-3 (main)
$ git add .
warning: in the working copy of 'praktikum-3/package.json', LF will be replaced by CRLF the next time Git touc
hes it
```

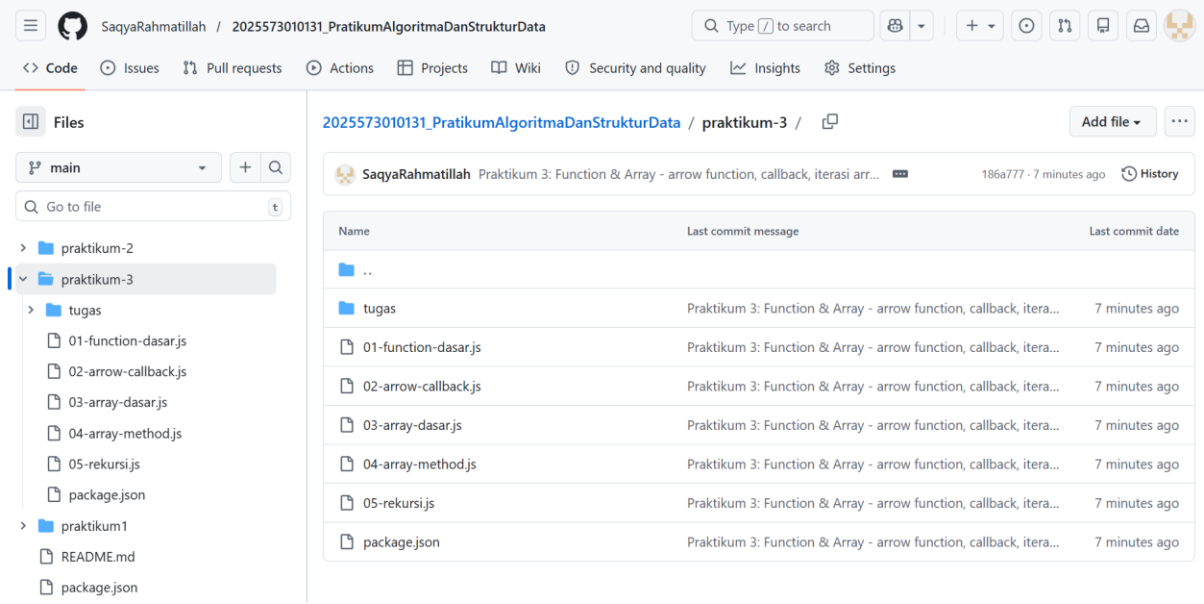
3. Lakukan commit dengan perintah:

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-3 (main)
$ git commit -m "Praktikum 3: Function & Array - arrow function, callback, iterasi array, rekursi"
git push origin main
[main 186a777] Praktikum 3: Function & Array - arrow function, callback, iterasi array, rekursi
 8 files changed, 466 insertions(+)
 create mode 100644 praktikum-3/01-function-dasar.js
 create mode 100644 praktikum-3/02-arrow-callback.js
 create mode 100644 praktikum-3/03-array-dasar.js
 create mode 100644 praktikum-3/04-array-method.js
 create mode 100644 praktikum-3/05-rekursi.js
 create mode 100644 praktikum-3/package.json
 create mode 100644 praktikum-3/tugas/tugas-1.js
 create mode 100644 praktikum-3/tugas/tugas-2.js
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 12 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (12/12), 5.25 KiB | 5.25 MiB/s, done.
Total 12 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/SaqyaRahmatillah/2025573010131_PratikumAlgoritmaDanStrukturData.git
   c135eda..186a777  main -> main
```

4. Unggah (push) project ke GitHub dengan perintah:

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-3 (main)
$ git push origin main
Everything up-to-date
```

5. Project berhasil di unggah (push).



BAB IV

HASIL DAN PEMBAHASAN

Berdasarkan praktikum yang telah dilakukan, seluruh konsep yang dipelajari pada modul ini dapat diimplementasikan dengan baik. Function yang dibuat mampu menerima parameter dan mengembalikan nilai sesuai dengan yang diharapkan. Hal ini menunjukkan bahwa mahasiswa sudah memahami konsep dasar function dengan cukup baik.

Penggunaan arrow function terbukti membuat penulisan kode menjadi lebih ringkas dan mudah dibaca, terutama untuk fungsi yang sederhana. Selain itu, callback function juga berhasil digunakan untuk menjalankan function lain sebagai parameter, sehingga program menjadi lebih fleksibel dan modular.

Pada bagian array, mahasiswa berhasil membuat dan mengolah data menggunakan berbagai method array. Method seperti *map* dan *filter* sangat membantu dalam memproses data tanpa perlu menggunakan perulangan manual. Hal ini membuat kode menjadi lebih efisien dan lebih mudah dipahami.

Selain itu, konsep rekursi juga memberikan pemahaman baru tentang bagaimana sebuah function dapat memanggil dirinya sendiri untuk menyelesaikan masalah tertentu. Meskipun konsep ini cukup sulit di awal, tetapi sangat berguna dalam pemrograman lanjutan.

Secara keseluruhan, praktikum ini memberikan pengalaman yang sangat baik dalam memahami bagaimana cara mengorganisir kode menggunakan function serta bagaimana mengolah data menggunakan array. Dengan pemahaman ini, mahasiswa akan lebih siap untuk mempelajari materi yang lebih kompleks.

BAB V

PENUTUP

5.1 Kesimpulan

Dari praktikum yang telah dilakukan, dapat disimpulkan bahwa function dan array merupakan konsep yang sangat penting dalam pemrograman JavaScript. Function membantu dalam membuat kode yang lebih terstruktur, sedangkan array memudahkan dalam pengolahan data.

Selain itu, penggunaan arrow function, callback, dan method array membuat program menjadi lebih efisien dan mudah dikembangkan. Dengan memahami konsep-konsep ini, mahasiswa memiliki dasar yang kuat untuk mempelajari pemrograman lebih lanjut.

5.2 Saran

Mahasiswa disarankan untuk terus berlatih dalam menggunakan function dan array agar semakin terbiasa dalam menulis kode. Selain itu, mahasiswa juga dapat mencoba membuat program yang lebih kompleks agar pemahaman semakin meningkat.

Selanjutnya, disarankan untuk mempelajari konsep lanjutan seperti object, asynchronous programming, dan struktur data lainnya agar kemampuan pemrograman semakin berkembang.